SECURITY ROADMAP

Building a Secure SaaS Product

From “it works” to “it is safe, observable, and resilient.”

Developer-focused | Secure by Design | Startup-ready



Prepared for engineering & product teams

Why security matters in SaaS

- Your product stores customer data, identity, and business workflows
- One auth or tenant-isolation bug can become a company-level incident
- Security reduces churn, sales friction, downtime, and legal/compliance risk
- Trust is part of the product experience — not a separate feature



Our security “north star”

Use known standards as guardrails — not as bureaucracy.

01

OWASP Top 10:2025

awareness of critical web risks

02

OWASP ASVS 5.0

testable app security requirements

03

NIST SSDF

secure software development practices

04

CIS Controls IG1

practical baseline cyber hygiene

05

MITRE ATT&CK

attacker behavior & threat modeling

Working rule: map every new feature to identity, access, data, abuse, and observability controls before merge.

What attackers target first

- Authentication and sessions
- APIs and object IDs
- Tenant boundaries
- Databases and storage
- CI/CD secrets and dependencies



Attack surface = every place where input, identity, data, or code crosses a trust boundary.

Common startup security mistakes

Hardcoded secrets	→	key leakage → account takeover
Frontend-only permission checks	→	IDOR / broken access control
No rate limits	→	brute force, scraping, spam
Public buckets / broad IAM	→	data exposure
No security logs	→	incident becomes guesswork
Rushed dependency updates	→	supply-chain risk

Core lesson: speed without guardrails creates invisible debt

How hackers attack a website



Most breaches are chains of small weaknesses — not one “Hollywood hack.”

OWASP Top 10:2025 — what matters for us

A01	Broken Access Control
A02	Security Misconfiguration
A03	Supply Chain Failures
A04	Cryptographic Failures
A05	Injection
A06	Insecure Design
A07	Authentication Failures
A08	Software/Data Integrity
A09	Logging & Alerting
A10	Exceptional Conditions

Priority order for SaaS: Access Control → Misconfiguration → Supply Chain → Auth → Logging

Authentication & session security



- Use secure cookies: HttpOnly, Secure, SameSite
- Rotate refresh tokens; revoke on suspicious activity
- Lock/rate-limit login, OTP, reset-password, and invite flows
- Never expose secrets or sensitive claims in frontend storage

Authorization & tenant isolation



- Object-level checks on every read/write
- Tenant ID must come from trusted server-side context
- Admin paths require separate privileges and audit logs

Rule: frontend visibility is UX — backend authorization is security

API security best practices

- Validate request schema and types
- Rate-limit by IP, user, tenant, and route sensitivity
- Restrict CORS; avoid wildcard origins with credentials
- Return safe errors; log full details server-side
- Set request body, upload, and pagination limits



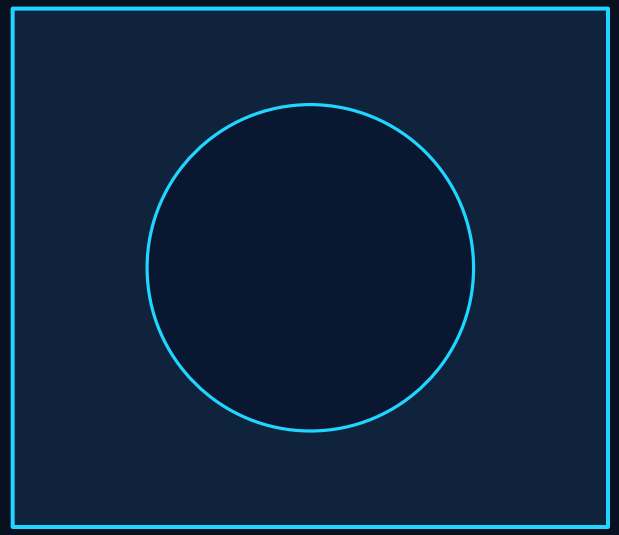
Data & database security

Protect the data lifecycle: collect less, store safely, access narrowly, recover reliably.



Database users should have role-specific permissions — application, migration, analytics, and admin should not share one superuser.

Secrets & configuration management



SECRET
VAULT

- Never commit .env files, tokens, private keys, or service credentials
- Use a secret manager; inject secrets at runtime
- Rotate keys after exposure, role changes, or vendor changes
- Separate dev, staging, and production credentials
- Least privilege for cloud IAM and third-party integrations

Pre-commit secret scan + protected CI secrets should be mandatory

Secure Development Lifecycle



Shift-left is not “more work for developers” — it means security feedback arrives before release pressure.

Cloud, DevOps & CI/CD security

- Protected branches + required reviews
- CI secrets masked and scoped
- SCA + container image scanning
- IaC review before cloud changes
- Least-privilege IAM for services
- Separate prod deployment approval

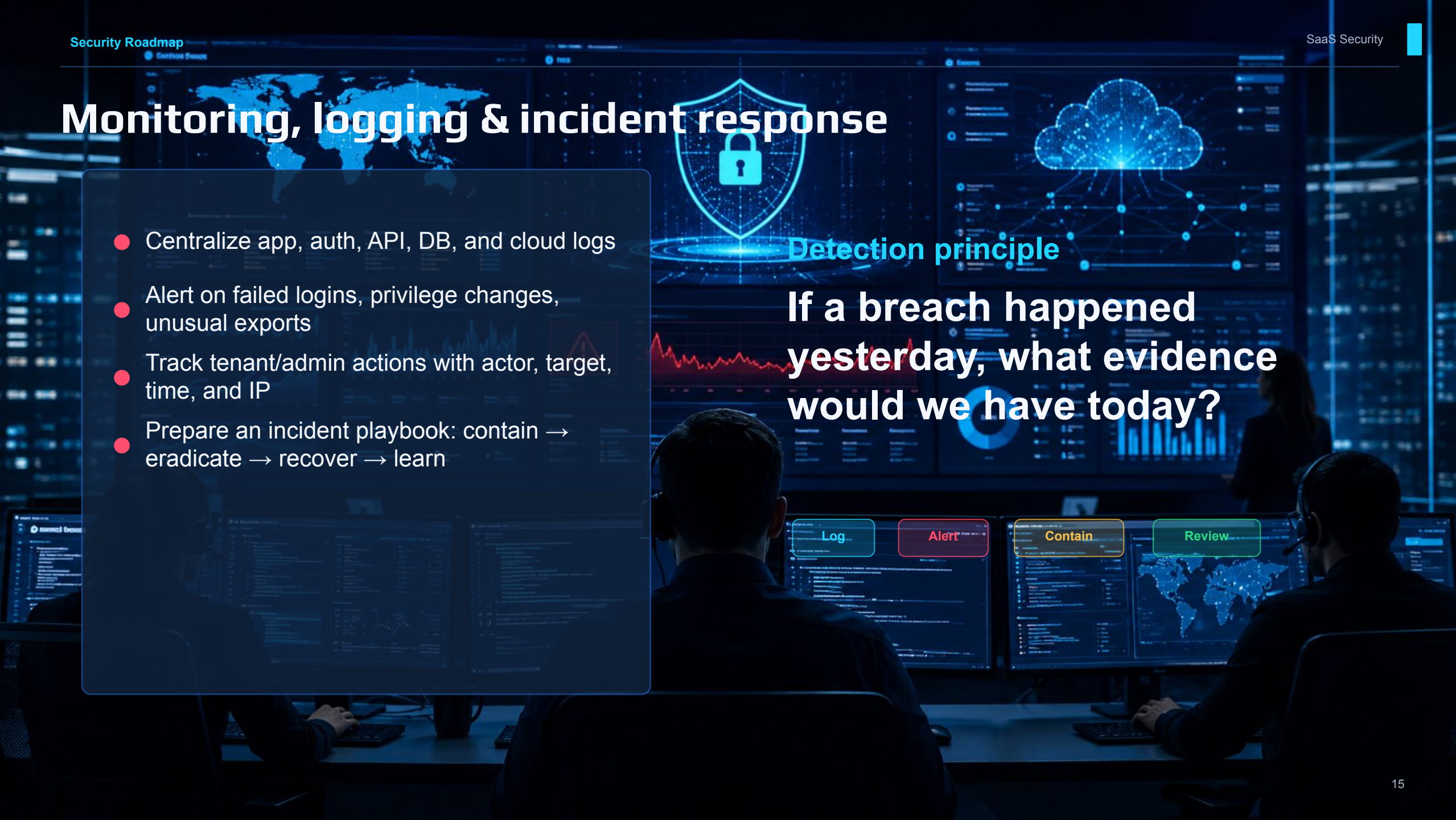


Monitoring, logging & incident response

- Centralize app, auth, API, DB, and cloud logs
- Alert on failed logins, privilege changes, unusual exports
- Track tenant/admin actions with actor, target, time, and IP
- Prepare an incident playbook: contain → eradicate → recover → learn

Detection principle

If a breach happened yesterday, what evidence would we have today?



90-day SaaS security roadmap

30 DAYS

Baseline

- Security audit
- Secrets cleanup
- Dependency scan
- Auth/API review

60 DAYS

Harden

- RBAC/tenant isolation
- Rate limiting
- Central logs
- Backup restore test

90 DAYS

Automate

- Threat modeling
- CI/CD gates
- Incident drill
- Pentest readiness

Final takeaway: every engineer owns security — but the product must make secure choices the default.